

All posts

2026

AAT intermittently unavailable via the Network of Terms

The Network of Terms in numbers: more than 11 million queries in 2026

A valid description isn't enough: the Dataset Register now goes deeper

Turning GeoNames into linked data

NDE releases SCHEMA-AP- NDE 1.0

Upcoming changes to the Requirements for Datasets

Is it <http://schema.org> or <https://schema.org>?

Search by subject in the Network of Terms

NDE services status page

Terminology source availability status

New thesaurus for movable agricultural heritage available via the Network of Terms

Can I use UUIDs with ARK?

New thesauri added to the Network of Terms

New Dataset Register website

2025

Is it <http://schema.org> or <https://schema.org>?

March 9, 2026 · 11 min read

David de Boer
Software Architect & Developer

Bob Coret

Schema.org is the web's **most widely adopted vocabulary for structured data**, supported by Google, Bing, and countless data platforms. It defines types like [CreativeWork](#), [Person](#), and [Organization](#), giving machines a common language to understand content. This is exactly why the [NDE application profile](#) [1](#) is built on Schema.org. Getting it right is what makes cultural heritage data findable and reusable across institutions.

But if you've worked with Schema.org in RDF, you've run into this: **there's <http://schema.org> and then there's <https://schema.org>**. That one-letter difference can cause real problems: SPARQL queries that silently return nothing, SHACL validation that rejects good data or ignores bad data, or datasets that should link up but don't – especially when combining data from multiple sources, as in NDE.

① UPDATE 2 APRIL 2026

This post originally recommended <http://schema.org/> (HTTP) based on reasons outlined below. It sparked exactly the discussion we hoped for: a structured conversation with stakeholders from NDE, CLARIAH, ODISSEI, and heritage institutions. On 2 April 2026, participants reached consensus on <https://schema.org/> (HTTPS). We acknowledge that this recommendation goes against one of the core principles of linked data – cool URIs don't change – but force of circumstance and pragmatic considerations have led us to advise HTTPS for Schema.org metadata going forward. This post has been updated to reflect that decision – see [the recommendation](#).

How did this happen?

Historically, Schema.org identifiers were defined using <http://schema.org/>. In 2018, as part of the [broader web push towards HTTPS](#) [2](#), the *website* moved to HTTPS. Schema.org's own [FAQ](#) [3](#) had already declared (since 2015 [4](#)) that "both 'https://schema.org' [5](#) and 'http://schema.org' [6](#) are fine." So people naturally started using <https://> – copying it from the site, its examples, and following the FAQ advice – even though the vocabulary identifiers were still defined as HTTP.

But a webpage is not the same as an identifier. Your browser just “ends up” at <https://schema.org/> either way, but for any tool that processes RDF, <http://schema.org/CreativeWork> and <https://schema.org/CreativeWork> are *two different things*.

As Tim Berners-Lee wrote in *Axioms of Web Architecture* [7](#): “the significance of identity for a given URI is determined by the person who owns the URI, who first determined what it points to.” So what did Schema.org's owner determine?

Why “both are fine” is misleading

Schema.org's [FAQ #19](#) [8](#) says that “both 'https://schema.org' [9](#) and 'http://schema.org' [10](#) are fine” and that “there should be no urgency about migrating existing data,” while noting the site itself has migrated to HTTPS as the default. That advice is accurate for search-engine consumers like Google, which normalize both variants. But for JSON-LD processors and RDF tooling, <http://schema.org/CreativeWork> and <https://schema.org/CreativeWork> are *not* the same – and the JSON-LD context still defines the vocabulary with HTTP identifiers.

JSON-LD contexts

JSON-LD is a common RDF serialization format. Schema.org uses it in its website examples. When you write Schema.org in JSON-LD, it looks like this:

```
{
  "@context": "https://schema.org",
  "@type": "CreativeWork"
}
```

JSON-LD processors follow a standardized mechanism called [remote document and context retrieval](#) [11](#):

- The processor sees `@context: "https://schema.org"`.
- It fetches that URL. The server responds with a `Link` header pointing to the JSON-LD context: `</docs/jsonldcontext.jsonld>; rel="alternate"; type="application/ld+json"`.
- The processor follows that link to <https://schema.org/docs/jsonldcontext.jsonld>.

Inside that document you'll find something important:

```
{
  "@context": {
    "@vocab": "http://schema.org/"
  }
}
```

This tells the JSON-LD processor: all Schema.org terms expand to <http://schema.org/> URIs. So even if your RDF uses <https://schema.org> as the context URL, a processor expands `"@type": "CreativeWork"` to <http://schema.org/CreativeWork>.

You can see this in action on the command line:

```
echo '{ "@context": "https://schema.org", "@type": "CreativeWork"; _:B... <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/name> "Example" }'
```

Tools – including structured-data validators, knowledge-graph pipelines, and libraries like Riot – normalize everything to HTTP URIs. They aren't downgrading security; they're following the vocabulary definition from the official context. As of Schema.org v29.4, this is still the case.

What about a custom context?

JSON-LD 1.1's [@import](#) [12](#) makes it possible to create a custom context that remaps everything to <https://schema.org/>:

```
{
  "@context": {
    "@version": 1.1,
    "@import": "https://schema.org/docs/jsonldcontext.jsonld",
    "@vocab": "https://schema.org/",
    "schema": "https://schema.org/"
  }
}
```

This overrides both `@vocab` and the `schema` prefix, so all terms expand to <https://schema.org/> while preserving type coercions (e.g. `url` and `sameAs` remain IRIs, not plain strings).

See [a shared context for consumers](#) for how NDE uses this.

The impact is limited

The JSON-LD context mismatch only affects tooling that processes JSON-LD and expands it to RDF triples. It does not affect:

- Data published in **Turtle, N-Triples, or other RDF serializations** – these use full URIs directly, so publishers simply write <https://schema.org/>.
- SPARQL queries and SHACL shapes** – these reference full URIs and are unaffected by JSON-LD context behaviour, *as long as* the data they operate on was not expanded from JSON-LD (where the context produces <http://> URIs).
- Search engines** like Google, which normalise both variants.

What does the community say?

The broader community has been debating this for years, with valid arguments on both sides.

Arguments for staying with HTTP:

- [schemaorg#4404](#) [13](#) (still open): contributors point out that “identifiers in the schema are technically URIs, not URLs. It's actually better if they don't change over time and there is no security implication involved in a predicate or type,” and that “Stable URIs that do not change over time are a MUST in the world of Linked Open Data.” Others describe the current situation as an “ugly intermediate zone” – neither fully HTTP nor fully HTTPS.
- [schemaorg#1325](#) [14](#) (closed, not planned): confirmed there would be no mandatory migration to HTTPS.
- [schemaorg#2597](#) [15](#): examples were updated to HTTPS, but the context was intentionally kept at HTTP.
- [schemaorg#2853](#) [16](#): a v12.0 attempt to switch the context to HTTPS was immediately reverted due to breakage.
- [ro-crate#427](#) [17](#) and [codemeta#373](#) [18](#): both RO-Crate and CodeMeta are discussing moving to HTTPS but still use HTTP.

Arguments for moving to HTTPS:

- [FAQ #19](#) [8](#) says “both are fine” and that HTTPS is “our preferred form in examples.”
- [rdflib](#) [19](#) defines its Schema.org namespace as <https://schema.org/> (exported as both `SDO` and `schema`).
- [PiCo](#) [20](#), an NDE community vocabulary, has already adopted HTTPS in practice.

Even prefix conventions reflect the divide: [prefix.cc](#) [21](#) maps `schema:` to <http://schema.org/> and `sdo:` to <https://schema.org/>.

What about security?

A common argument for HTTPS identifiers is security: shouldn't we fetch vocabulary definitions over a secure connection? In practice, this is a non-issue. If you dereference <http://schema.org/CreativeWork>, the server redirects you to <https://schema.org/CreativeWork> – the definition is served over HTTPS regardless. The HTTP in the URI is the *identifier*, not the transport protocol.

What about dereferenceability?

Schema.org URIs aren't just opaque strings – they *do* resolve to useful definitions, and that dereferenceability is a core Linked Data principle. You can look up what [CreativeWork](#) means, what properties it has, and how it relates to other types. This remains fully functional with HTTP identifiers: you still get the definitions, still over a secure connection. What matters for interoperability is that everyone uses the *same* identifier, whichever scheme it uses.

Cool URIs don't change

Tim Berners-Lee wrote in his essay *Cool URIs don't change* [22](#): URIs should be stable over time. Once <http://schema.org/CreativeWork> became the identifier for that concept, changing it to <https://schema.org/CreativeWork> would break existing data, queries, and validation rules.

He recently reflected on the HTTP-to-HTTPS transition in his book *This is for Everyone* [23](#) (p. 176):

“We hit an unfortunate snag when we changed the URL to add an ‘S’: it broke all existing links on the web! In retrospect, the web would be much more functional and simpler if HTTP and HTTPS pages both used the same URL, just with different protocols.”

The URI scheme change should not have affected identity – but it did. Using HTTP for vocabulary identifiers is not unusual: Dublin Core Terms, for example, uses <http://purl.org/dc/terms/> and has never changed to HTTPS.

The recommendation

On 2 April 2026, representatives from NDE, research infrastructures (CLARIAH, ODISSEI), and heritage institutions (IISG and others) reached consensus: **use <https://schema.org/>**.

We don't take this lightly. Changing the URI for an established vocabulary goes against [Cool URIs don't change](#), one of the core principles of linked data. Still, by force of circumstance and on the strength of the pragmatic arguments below, we recommend HTTPS for Schema.org metadata.

The arguments that tipped the balance:

- Future-proof.** Schema.org's own [FAQ](#) [24](#) signals HTTPS as the preferred direction. Choosing HTTP now would risk a second migration if CLARIAH, ODISSEI, and others change.
- Easier to communicate.** Telling institutions to use <http://> – while their browsers show <https://> – invites confusion and requires explaining the identifier-vs-transport distinction every time.
- Network effect.** Vocabularies like [PiCo](#) [25](#) and tools like [rdflib](#) [19](#) already use <https://schema.org/>. Aligning with them strengthens interoperability across NDE, CLARIAH, ODISSEI, and beyond.

The [NDE Schema Application Profile](#) [26](#) now recommends:

- New datasets** MUST use <https://schema.org/> URIs.
- Existing datasets** SHOULD migrate to <https://schema.org/>.
- Consumers** MUST accept both <http://> and <https://> variants. NDE will publish using [https://](https://schema.org/) data from sources outside the NDE network. This is another reason why consumer-side normalisation matters.

This follows the [robustness principle](#) [27](#): be strict in what you publish, liberal in what you accept.

A shared context for consumers

For the JSON-LD case, rather than requiring every data publisher to adopt a custom context, **the responsibility sits with consumers**. Publishers keep using the standard `"@context": "https://schema.org"` – no extra dependencies. Consumers apply a shared context when parsing incoming JSON-LD data, rewriting <http://> terms to <https://> at ingestion time.

NDE provides [this context](#) [28](#) using the [@import](#) mechanism described above. Consumers replace the context of incoming documents before expansion:

```
// Node (jsonld.js)
const ['@context'] = "https://docs.nde.nl/schema-profile/document" expanded = await jsonld.expand(document);
```

```
# Python (pyLd)
document["@context"] = "https://docs.nde.nl/schema-profile/" expanded = jsonld.expand(document)
```

This approach is **stack-independent**: one URL that works in any JSON-LD 1.1 processor, regardless of language or platform. No per-stack stream transforms needed. Alternatively, consumers can [inline the context](#) to avoid the external dependency.

Upstream dependencies

Some upstream standards like [RO-Crate](#) [29](#) and [CodeMeta](#) [30](#) still use <http://schema.org/> URIs. This doesn't affect the NDE recommendation directly – our [Requirements for Datasets](#) [31](#) already use <https://> – but it means consumers may still encounter <http://> data from sources outside the NDE network. This is another reason why consumer-side normalisation matters.

Conclusion

<https://schema.org/> is the canonical form for Schema.org URIs in the Dutch cultural heritage network. That recommendation comes with an honest caveat: it goes against [Cool URIs don't change](#). We make it only because the alternative – an indefinite split between <http://> and <https://> across the network – does more day-to-day harm to interoperability than holding the line on the principle would prevent. The [JSON-LD context mismatch](#) is a real but limited issue, addressed by a [shared consumer-side context](#). When publishing structured data: use <https://schema.org/>. When consuming it: accept both and normalise to <https://>. And for institutions whose graphs make migration a significant project: take the time you need – existing <http://schema.org/> data is not wrong, and the network is built to accept it.

This post was updated following a community decision on 2 April 2026. The original version recommended <http://schema.org/> – the updated recommendation is <https://schema.org/>. Thanks to participants from heritage institutions, research infrastructures, and academia – and to Ivo Zandhuis, Richard Zijdemán, and Leon van Wissen for their early feedback.

Tags: [RDF](#)

[Edit this page](#) Last updated on **Apr 2, 2026**

Upcoming changes to the Requirements for Datasets

Search by subject in the Network of Terms

How did this happen?

Why “both are fine” is misleading

JSON-LD contexts

What about a custom context?

The impact is limited

What does the community say?

What about security?

What about dereferenceability?

Cool URIs don't change

The recommendation

A shared context for consumers

Upstream dependencies

Conclusion